Agentic AI Engineering

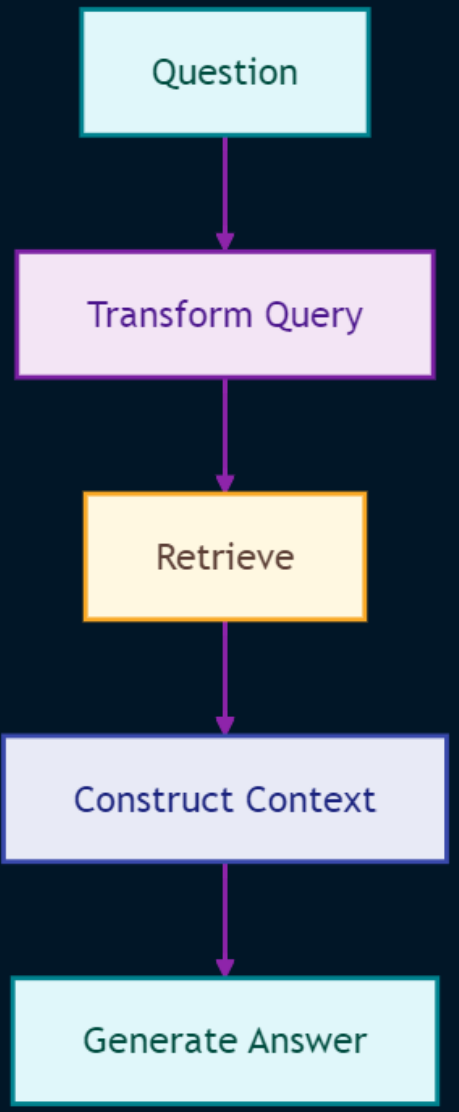
Adaptive Query Transformation for Advanced RAG Retrieval

Max Montaseri

- Designed an advanced **query-transformation RAG architecture** using LangChain and Chroma to improve semantic retrieval for poorly worded, ambiguous, multi-intent, and highly specific user questions.
- Implemented a **Rewrite-Retrieve-Read** pipeline that uses an LLM to transform natural-language questions into retrieval-optimized semantic search queries while preserving the original question for answer generation.
- Built a custom **MultiQueryRetriever** workflow generating five alternative query perspectives and implemented a custom BaseOutputParser to convert LLM output into structured query lists.
- Implemented **Step-Back Question Retrieval** to transform detailed questions into broader information needs and retrieve higher-level context for more comprehensive LLM responses.
- Implemented **HyDE-based retrieval**, generating hypothetical answer-like documents from user questions and using the generated representation to improve document retrieval.
- Applied **LCEL composition** with RunnablePassthrough, prompt templates, retrievers, ChatOpenAI, and output parsers to construct modular RAG pipelines.
- Built a structure-aware tourism knowledge ingestion pipeline using AsyncHtmlLoader, HTMLSectionSplitter, OpenAI embeddings, and Chroma vector storage.
- Established an extensible foundation for adaptive retrieval, reranking, hybrid search, retrieval evaluation, and LangGraph-based conditional orchestration.

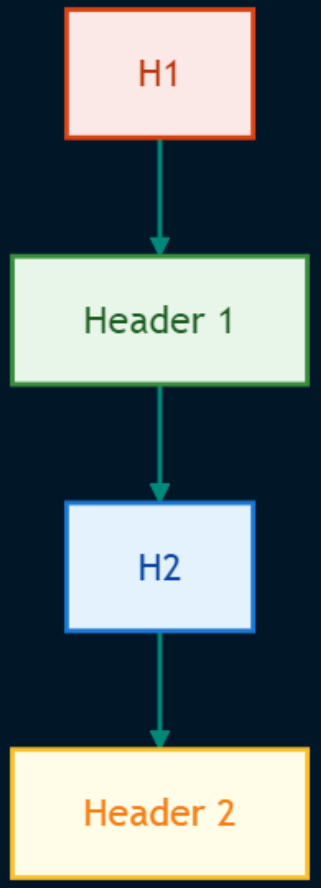
Architecture

The four strategies share a common conceptual architecture

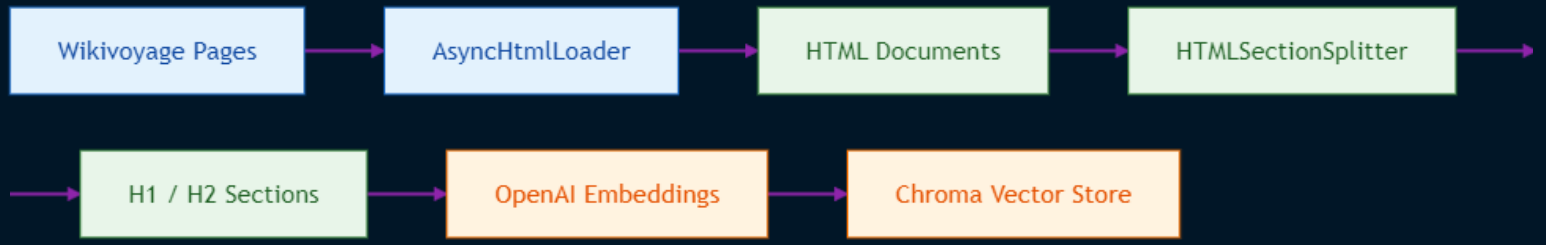


Ingestion Pipeline

The project uses AsyncHtmlLoader to retrieve the tourism pages and HTMLSectionSplitter to preserve HTML heading structure.



Architecture



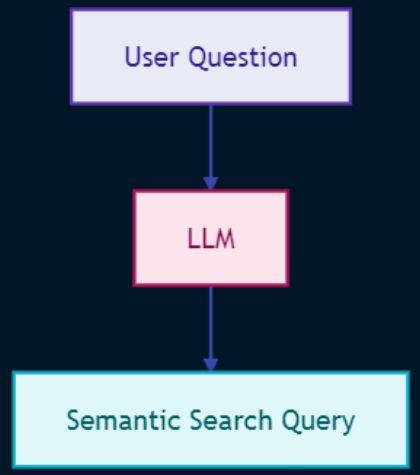
Strategy 1: Rewrite-Retrieve-Read

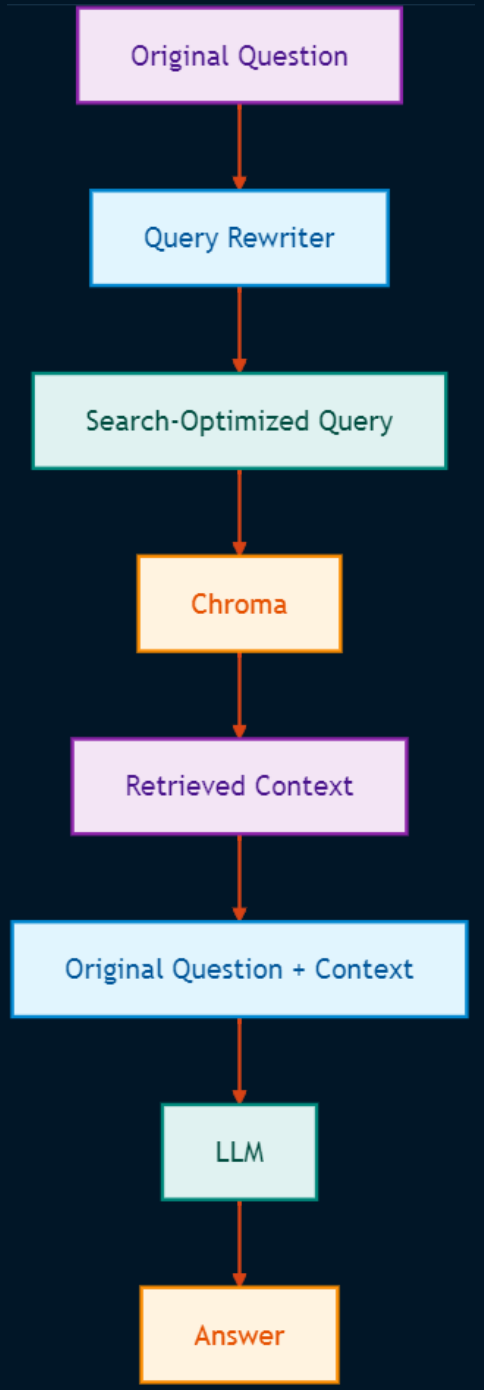
The first strategy uses an LLM to transform a user's original question into a query optimized for semantic vector search.

The source first performs direct similarity search and observes that the retrieval quality for the original tourism question is poor.

Query Rewriter

The rewriter prompt instructs the LLM to generate a search query suitable for Chroma and return only the revised query.

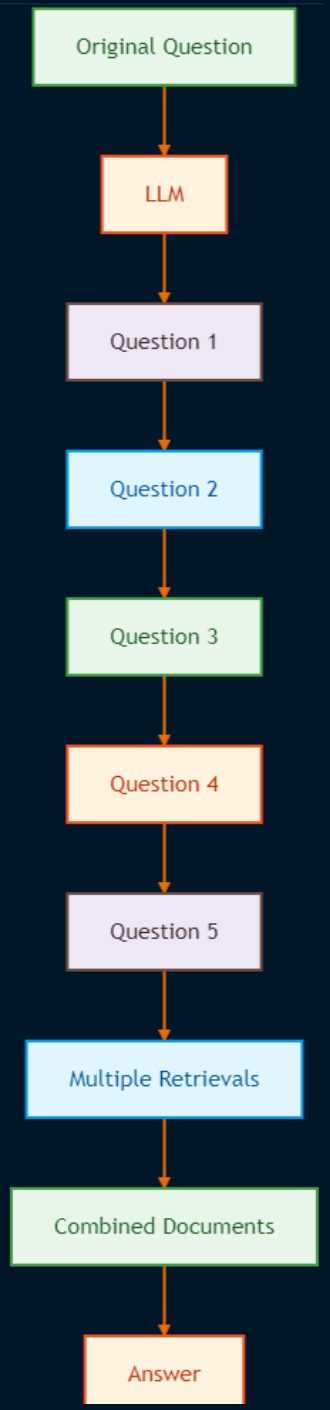




Strategy 2: Multi-Query Retrieval

Problem
A single embedding of a complex question may represent only one semantic interpretation.
The source explains that a well-formed question can contain multiple implicit questions, making a single rewritten query insufficient.

Solution
Generate multiple alternative questions.



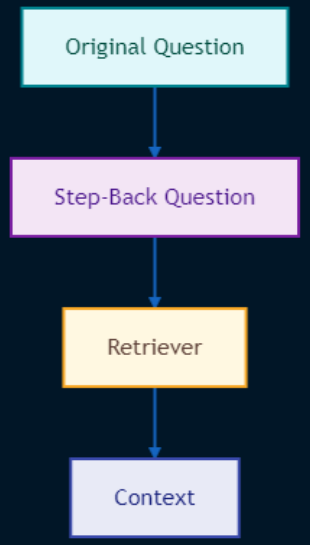
Strategy 3: Step-Back Questions

Problem
Highly detailed questions can retrieve overly narrow chunks. The source explains that detailed retrieval may miss broader contextual information.

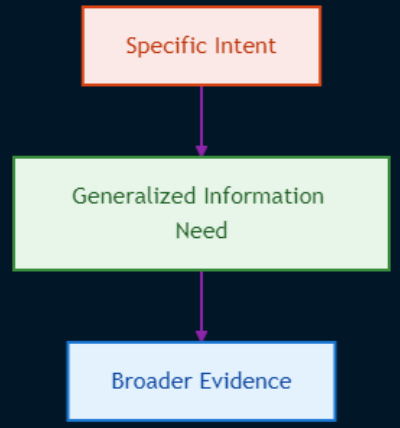
Solution
Generate a broader question from the detailed question.

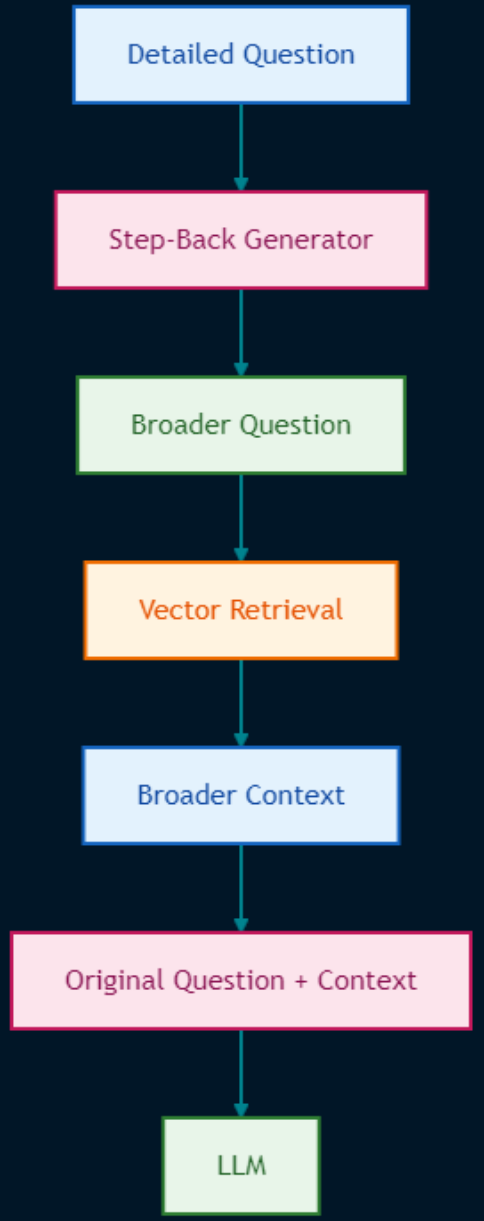
Prompt
The step-back prompt asks the model to generate a less-specific question so wider context can be retrieved.

RAG Integration



Important Architectural Concept
Step-back retrieval is a form of **query abstraction**:

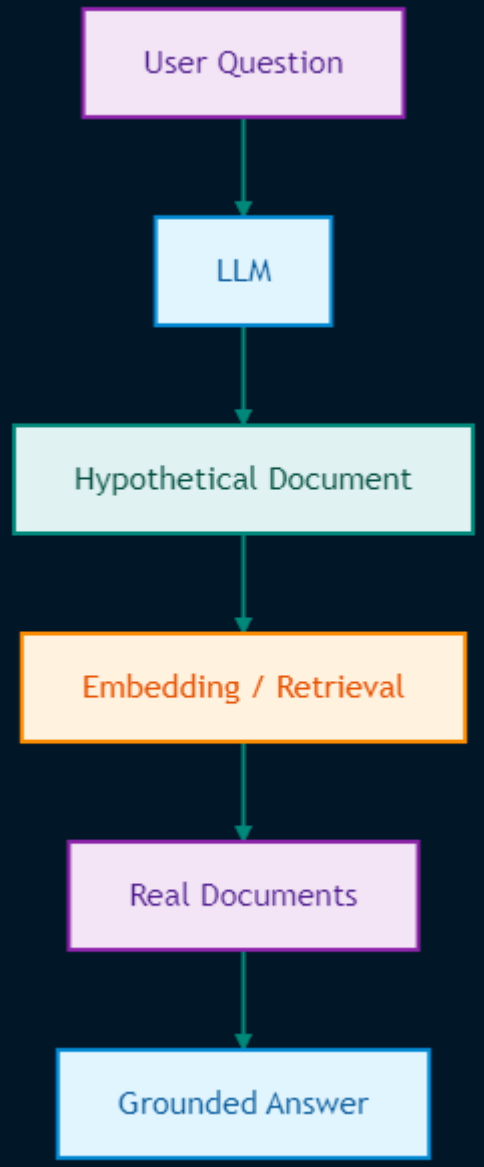




Strategy 4: HyDE

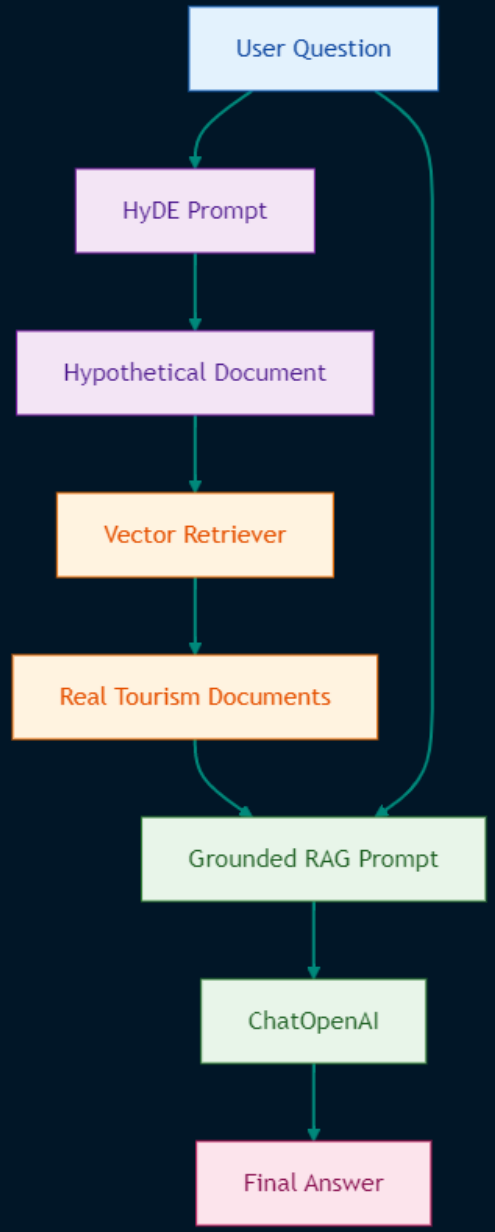
Hypothetical Document Embeddings (HyDE) changes the retrieval representation differently. Instead of modifying the indexed document, the system asks the LLM to generate a hypothetical answer-like document.

The source distinguishes HyDE from hypothetical-question indexing: HyDE keeps the original chunk embeddings unchanged and generates a hypothetical document from the user's question.



HyDE RAG Flow

The source integrates the generated hypothetical document into retrieval and retains the original question for final generation.

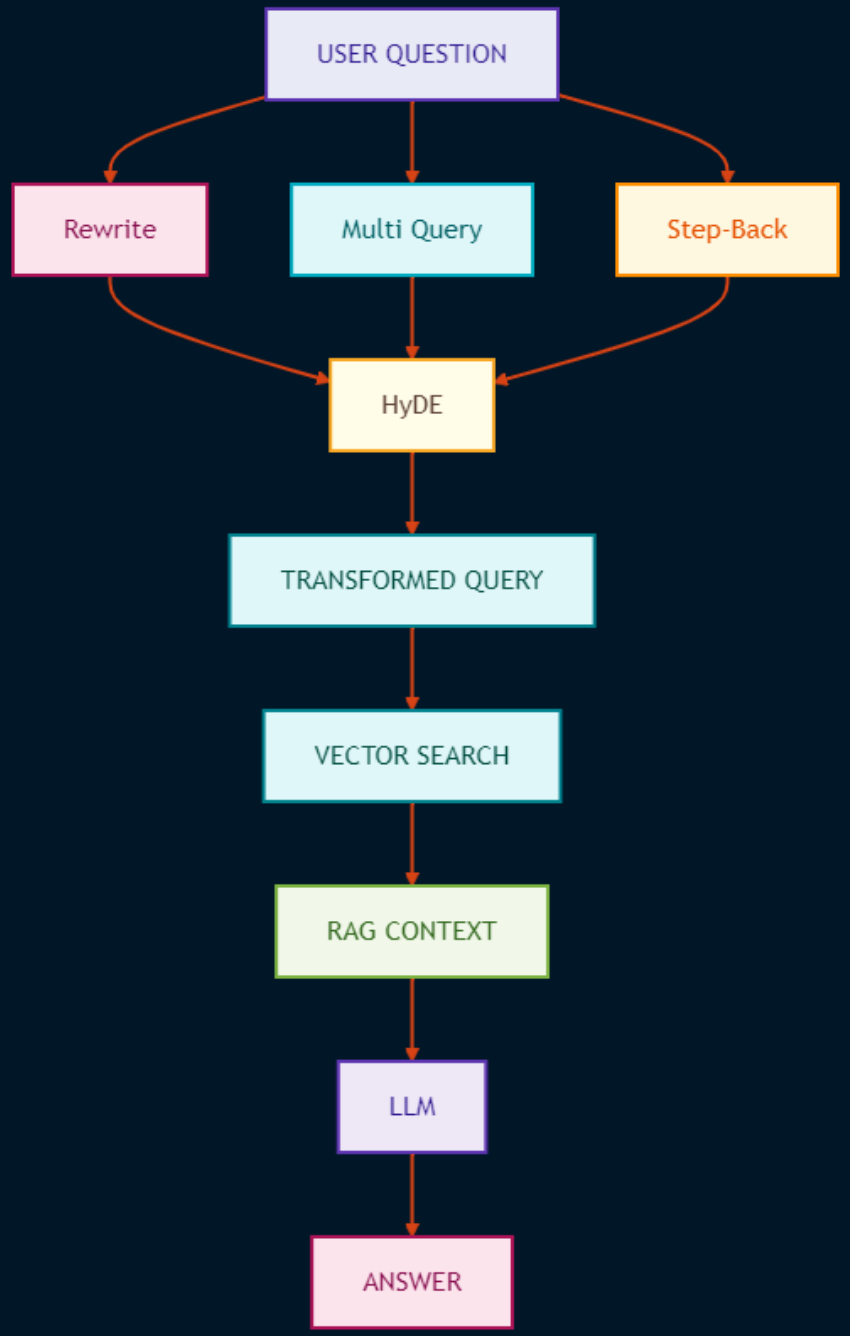


Strategy Comparison

Strategy	Transformation	Retrieval Objective	Main Advantage
Rewrite-Retrieve-Read	One optimized query	Improve semantic query wording	Simple and efficient
Multi-Query	Multiple query variants	Capture multiple interpretations	Broader recall
Step-Back	Broader question	Retrieve higher-level context	Better contextual coverage
HyDE	Hypothetical answer/document	Align query with document semantics	Different query-document representation

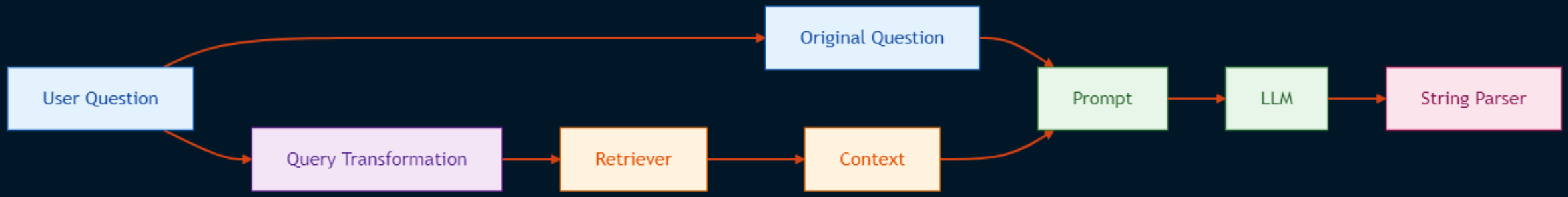
Query Transformation Architecture

All four strategies can be viewed as variations of the same architectural principle



LangChain Expression Language (LCEL) Design

The larger RAG chain composes parallel input preparation, retrieval, prompting, LLM generation, and parsing.

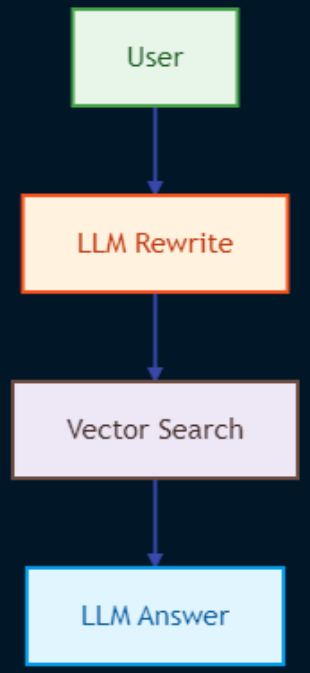


Performance and Cost

Query transformation introduces additional LLM inference before vector retrieval.

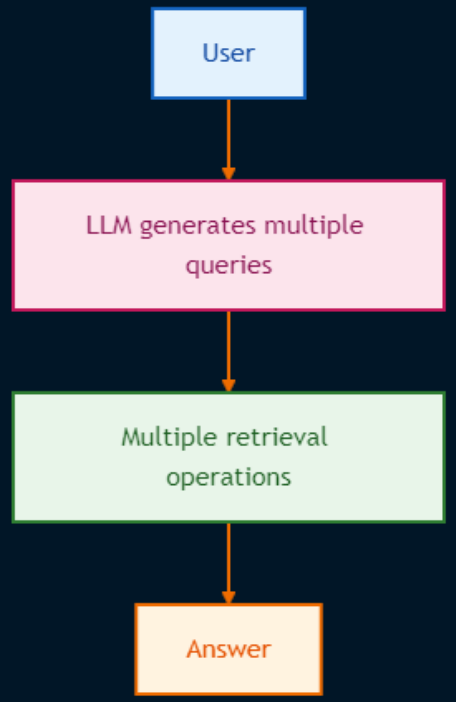
Rewrite

This adds one LLM call but normally performs one retrieval.



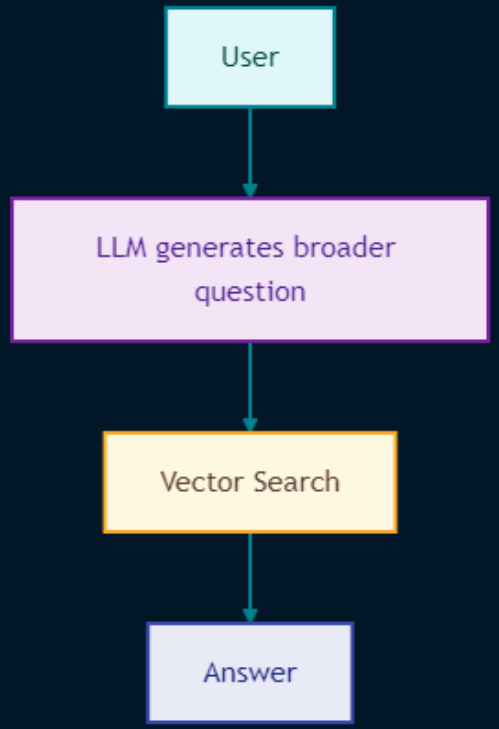
Multi-Query

This increases retrieval workload and potentially increases context volume.



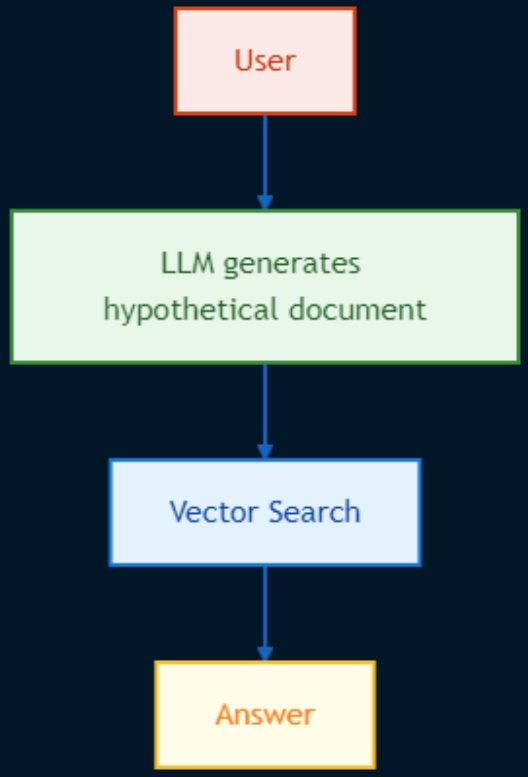
Step-Back

The project demonstrates a single step-back retrieval path.



HyDE

This adds an LLM call before retrieval.



Engineering Trade-Off

Query transformation should not automatically be enabled for every request.

A production architecture could route requests based on:

- question complexity
- retrieval confidence
- query length
- ambiguity
- domain
- latency budget

Production Roadmap

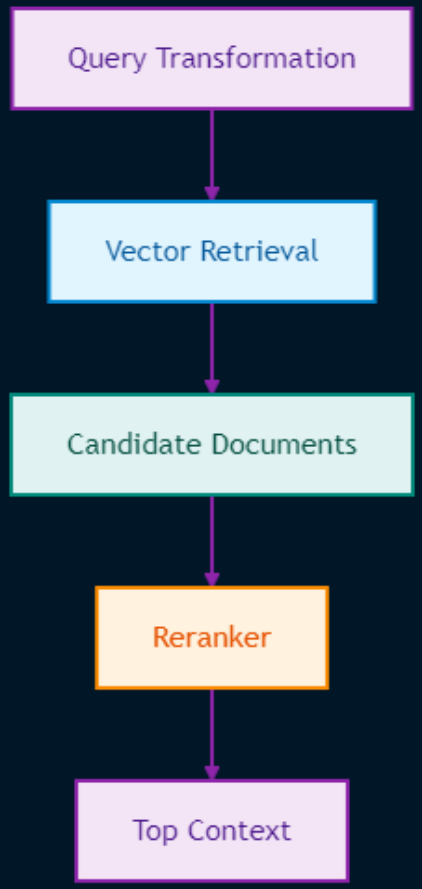
Retrieval Evaluation

Build a labeled benchmark:

Question Expected Relevant Documents Expected Evidence Expected Answer

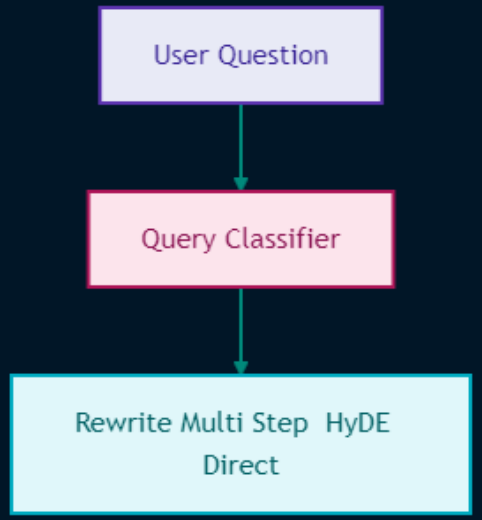
Evaluate each strategy independently.

Add Reranking



Adaptive Query Transformation

Instead of running every transformation



Hybrid Retrieval

Combine:

Vector Search
+
Keyword Search
+
Metadata Filtering
+
Reranking

Observability

Capture:

- transformation strategy
- original query
- transformed query
- retrieved document IDs
- retrieval scores
- latency
- token usage
- final answer
- evaluation score

Production Roadmap

LangGraph Orchestration

A production Agentic AI evolution could model query transformation as conditional graph nodes.

This diagram represents a **recommended future architecture**, not functionality currently implemented in the source.